



UNIVERSITATEA DIN
BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

PLATFORMĂ BAZATĂ PE DOVEZI
PENTRU MONITORIZAREA
ANTRENAMENTELOR,
DESCOPERIREA DE PROGRAME,
COACHING ȘI CĂUTARE ÎN
ISTORICUL DE ANTRENAMENT

Absolvent

[Prenume NUME]

Coordonator științific

[Titlul și numele coordonatorului]

București, iulie 2026

Rezumat

În această lucrare prezint o aplicație web pentru antrenamente de forță, construită în jurul unei probleme care apare destul de repede într-un astfel de sistem: istoricul de antrenament trebuie să rămână corect în timp. Când utilizatorul efectuează un antrenament, planul de la care a pornit este copiat („înghețat”) sub formă de instantanee. Astfel, dacă mai târziu utilizatorul modifică sau șterge datele inițiale, ceea ce a fost deja înregistrat rămâne neschimbat. Aplicația include gestionarea programelor de antrenament, executarea sesiunilor, urmărirea progresului, un marketplace de programe (publicare, vot, salvare, copiere), o analiză a structurii programului bazată pe reguli simple și explicabile (pornite din literatura de specialitate, fără pretenții medicale), un mod de coaching în care un antrenor poate vedea datele unui client doar cu acordul acestuia, precum și o căutare full-text peste programe și peste istoric. Căutarea este tratată ca un model de citire separat, construit peste OpenSearch și actualizat prin evenimente declanșate după salvarea datelor în baza de date. O parte importantă a lucrării este comparația practică dintre OpenSearch și PostgreSQL, realizată pe un set de date generat și crescut până la 2 milioane de documente. Backend-ul este scris în Spring Boot 3 / Java 21, cu PostgreSQL 16 ca sursă unică de adevăr, iar corectitudinea este verificată prin teste de integrare pe baze de date reale (Testcontainers, fără mock-uri). Rezultatele arată că cele două soluții se echivalează în jurul a 10.000 de documente și că diferența cea mai clară apare la căutarea care tolerează greșelile de tastare.

Abstract

This thesis presents a web application for strength training, built around a problem that appears quite naturally in this type of system: the training history has to stay correct over time. When a workout is performed, the plan it started from is copied (“frozen”) as snapshots. This means that if the user later edits or deletes the original data, the information already recorded in the history does not change. The application includes training program management, live workout sessions, progress tracking, a program marketplace (publish, vote, save, copy), an analysis of program structure based on simple and explainable rules (derived from the training literature, with no medical claims), a coaching mode where a coach can view a client’s data only with that client’s consent, and full-text search over both programs and workout history. Search is implemented as a separate read model, built on top of OpenSearch and updated through events fired after the data is saved to the database. A central part of the thesis is a practical comparison between OpenSearch and PostgreSQL, using a generated data set scaled up to 2 million documents. The backend is written in Spring Boot 3 / Java 21, with PostgreSQL 16 as the single source of truth, and correctness is checked through integration tests that run against real databases (Testcontainers, no mocks). The results show that the two options are roughly equivalent around 10,000 documents, while the clearest difference appears in typo-tolerant (fuzzy) search.

Cuprins

1	Introducere	6
1.1	Tipul lucrării și încadrarea temei	6
1.2	Prezentarea generală a temei	6
1.3	Scopul și motivația alegerii temei	7
1.4	Contribuția proprie (pe scurt)	7
1.5	Structura lucrării	8
2	Preliminarii	9
2.1	Tehnologii de bază	9
2.2	Regăsirea informației și căutarea full-text	9
2.3	Modele de citire separate și consistență eventuală	10
2.4	Imuabilitatea istoricului	10
2.5	De unde vin regulile analizorului	11
2.6	Obiectivele lucrării	11
3	Contribuția proprie	12
3.1	Cerințe și prezentare generală a sistemului	12
3.2	Arhitectura sistemului	12
3.3	Modelul de date și principiul instantaneelor	13
3.4	Modelul de securitate	14
3.5	Funcționalitățile aplicației	15
3.6	Analizorul structural informat de dovezi	16
3.7	Modelul de citire pentru căutare (OpenSearch)	16
3.8	Evaluarea empirică: OpenSearch vs. PostgreSQL	18
3.9	Modul de coaching: RBAC plus ReBAC	19
3.10	Strategia de testare	19
3.11	Direcții de dezvoltare proiectate	19
4	Concluzii	21
4.1	Concluzii privind realizarea temei	21
4.2	Aprecieri personale privind relevanța rezultatelor	21

4.3 Dezvoltări ulterioare	22
Bibliografie	23
Anexa 1 — Capturi de ecran ale interfeței	25
Anexa 2 — Schema bazei de date	28
Anexa 3 — Listări de cod reprezentative	29
Anexa 4 — Rezultatele complete ale evaluării	31
Anexa 5 — Contractul API (selecție)	33
Anexa 6 — Configurație și rulare	34

Capitolul 1

Introducere

1.1 Tipul lucrării și încadrarea temei

Lucrarea de față este o lucrare de licență aplicativă, încadrată în zona aplicațiilor web full-stack. Pe lângă partea obișnuită de inginerie software (baze de date, securitate, testare), tema include și o componentă de regăsire a informației: o căutare full-text construită peste motorul OpenSearch. Din punctul meu de vedere, partea cea mai interesantă este comparația dintre această variantă și ceea ce poate oferi PostgreSQL fără un motor separat de căutare.

1.2 Prezentarea generală a temei

Aplicațiile pentru monitorizarea antrenamentelor ajung, de multe ori, în două zone destul de diferite. Unele sunt mai ales jurnale: memorează ce a făcut utilizatorul, dar oferă puțin sprijin pentru planificare. Altele încearcă să se prezinte ca un fel de „antrenor cu inteligență artificială”, cu recomandări greu de verificat de utilizator și care pot părea mai sigure decât sunt de fapt. În plus, istoricul este uneori tratat ca și cum ar fi doar o extensie a datelor curente: dacă programul inițial este modificat sau șters, antrenamentele vechi pot deveni neclare sau chiar incoerente.

Platforma realizată în această lucrare pornește de la o idee diferită. În primul rând, istoricul este tratat ca o înregistrare care nu ar trebui rescrisă: la executarea unui antrenament, planul este copiat în instantanee, iar datele istorice rămân corecte indiferent ce se întâmplă ulterior cu datele sursă. În al doilea rând, feedback-ul oferit utilizatorului este explicabil: analiza se bazează pe reguli clare, inspirate din literatura despre antrenamentul de forță, fără a pretinde că oferă sfat medical. În al treilea rând, căutarea nu se oprește la un filtru SQL simplu, ci folosește un model de citire construit special pentru căutare, iar lucrarea măsoară concret când începe să merite această alegere.

1.3 Scopul și motivația alegerii temei

Scopul meu a fost să construiesc un sistem cât mai apropiat de unul real, nu doar o aplicație de laborator. Am urmărit să acopăr câteva aspecte importante din ingineria software: securitate (autentificare cu token-uri, control al accesului pe roluri și pe relații), integritatea datelor (instantanee care nu se modifică, constrângeri la nivelul bazei de date), modele de citire separate și o testare cât mai apropiată de mediul real.

Am ales căutarea ca temă centrală din două motive. Pe de o parte, o căutare full-text care acceptă greșeli de tastare, ordonează rezultatele după relevanță și oferă fațete este utilă într-un istoric mare de antrenamente, de exemplu atunci când utilizatorul vrea să găsească un antrenament vechi din 2020. Pe de altă parte, decizia de a folosi doar baza de date sau de a adăuga un motor de căutare separat apare des în proiecte, dar nu este întotdeauna justificată prin măsurători. În lucrare am încercat să formulez această alegere ca o întrebare măsurabilă: *de la ce dimensiune începe să merite un motor de căutare dedicat?*

1.4 Contribuția proprie (pe scurt)

Am proiectat și implementat singur întregul sistem. Contribuțiile principale sunt:

- un model de date orientat spre integritate (instantanee care nu se modifică, ștergere logică, indecși unici parțiali, constrângeri `CHECK`), validat de Hibernate în modul `validate`;
- un sistem de securitate cu token-uri JWT și rotația token-urilor de reîmprospătare, cu acces controlat pe roluri (RBAC) și pe relații (ReBAC) pentru coaching, proiectat astfel încât să evite atacurile de tip IDOR (răspunde cu 404, nu cu 403);
- un analizor al structurii programelor, bazat pe reguli explicabile, cu un scor pe 100 de puncte împărțit în cinci părți și cu avertismente motivate;
- un model de citire separat pentru căutare, construit peste OpenSearch și actualizat prin evenimente după salvare, cu securitate aplicată pe două straturi;
- o comparație practică între OpenSearch și PostgreSQL pe un set de date generat, crescut de la 1.000 la 2.000.000 de documente, în care varianta PostgreSQL folosește mecanismele potrivite pentru această problemă (`tsvector`+GIN, `pg_trgm`);
- o suită de teste de integrare care rulează pe baze de date reale (PostgreSQL și OpenSearch prin Testcontainers), fără mock-uri și fără H2.

1.5 Structura lucrării

Lucrarea este organizată astfel. Capitolul 2 (*Preliminarii*) introduce noțiunile și tehnologiile pe care se sprijină tema, precum și contextul în care se încadrează. Capitolul 3 (*Contribuția proprie*) descrie arhitectura sistemului, modelul de date și instanțele, securitatea, analizorul, căutarea peste OpenSearch, comparația OpenSearch vs. PostgreSQL, modul de coaching, testarea și direcțiile deja proiectate pentru viitor. Capitolul 4 (*Concluzii*) sintetizează rezultatele și discută pașii următori. Detaliile mai voluminoase — capturi din interfață, schema completă a bazei de date, fragmente de cod, tabelele cu toate măsurătorile și lista de endpoint-uri — sunt puse în anexe și sunt referite din text.

Capitolul 2

Preliminarii

În acest capitol prezint pe scurt tehnologiile și ideile pe care se bazează lucrarea, contextul în care se află tema și obiectivele pe care mi le-am propus pornind de aici.

2.1 Tehnologii de bază

Backend. Partea de server este scrisă în Java 21 cu Spring Boot 3 [14] și folosește module pentru web (REST), securitate, validare și acces la date prin Spring Data JPA / Hibernate. Datele sunt păstrate în PostgreSQL 16 [11], iar schema este gestionată doar prin migrări Flyway. Un detaliu important este modul `ddl-auto: validate`: Hibernate nu creează și nu modifică schema, ci verifică la pornire că entitățile corespund structurii din baza de date. Astfel, nepotrivirile apar imediat, nu abia în timpul rulării aplicației.

Frontend. Interfața este o aplicație single-page scrisă în React cu TypeScript [6]. Ea este servită de backend, de la aceeași adresă, lucru care simplifică folosirea cookie-urilor și configurarea de securitate.

Autentificare. Pentru autentificare folosesc *JSON Web Tokens* (JWT) [4]: un token de acces semnat, cu durată scurtă, trimis în antetul `Authorization`, și un token de reîmprospătare opac, pentru care în baza de date se păstrează doar o amprentă. Parolele sunt stocate tot ca amprente, generate cu o funcție potrivită pentru rezistență la atacuri prin dicționar.

2.2 Regăsirea informației și căutarea full-text

Regăsirea informației (information retrieval) se ocupă de găsirea documentelor relevante pentru o anumită căutare. Spre deosebire de o potrivire exactă în SQL, căutarea full-text implică mai multe etape: împărțirea textului în termeni și normalizarea lui (inclusiv scoaterea diacriticelor), un *index inversat* care leagă fiecare termen de documentele

în care apare, o funcție de scor care ordonează rezultatele după relevanță și, în multe cazuri, un mecanism pentru tolerarea greșelilor.

Relevanță și ordonare. Motoarele moderne folosesc frecvent funcția de scor *BM25* [9], din familia TF-IDF, dar mai potrivită pentru colecții reale de documente, deoarece ține cont atât de frecvența termenilor, cât și de lungimea documentului. Calitatea ordonării poate fi măsurată cu indicatori precum *precision@k* și *nDCG* [3], care pun accent pe apariția rezultatelor relevante cât mai sus în listă.

Toleranța la greșeli. Căutarea „fuzzy” acceptă potriviri aproximative, de obicei pe baza distanței de editare (Levenshtein). Ea este utilă pentru greșeli de tastare, de exemplu *benhc* → *bench*. În bazele de date relaționale, un efect asemănător se poate obține cu extensia *pg_trgm*, care compară cuvintele prin trigrame.

Căutarea în PostgreSQL vs. motoare dedicate. PostgreSQL poate face căutare full-text prin tipul *tsvector*, funcții de ordonare precum *ts_rank_cd* și indecși GIN [12]. Pentru volume mici, această variantă poate fi suficientă și are avantajul că nu adaugă încă o componentă în sistem. Motoarele dedicate, cum este OpenSearch [7] (o variantă open-source a Elasticsearch, construită peste biblioteca Lucene), oferă în schimb analizoare configurabile, sinonime, scoruri BM25, fațete prin agregări și căutare tolerantă la greșeli care rămâne rapidă la volume mari. Din acest motiv, alegerea depinde mult de scară, iar în lucrare am verificat experimental acest lucru.

2.3 Modele de citire separate și consistență eventuală

Pentru a separa scrierea datelor (tranzacțională și normalizată) de citirea optimizată pentru interogări, am folosit un *model de citire separat*, construit dintr-o singură sursă de adevăr. Se poate privi ca o formă mai relaxată a tiparului CQRS [5]. Modelul separat, în cazul meu indexul OpenSearch, este *eventual consistent* cu sursa de adevăr (PostgreSQL) și poate fi reconstruit oricând. Dificultatea principală este propagarea sigură a modificărilor către modelul separat, adică problema *dual-write*, discutată în Capitolul 3.

2.4 Imuabilitatea istoricului

Ideea că datele istorice trebuie tratate ca fapte care nu se rescriu este cunoscută în sistemele care lucrează cu volume mari de date [2]. În aplicația mea, un antrenament efectuat este un astfel de fapt: nu ar trebui să se schimbe doar pentru că planul din care a pornit a fost editat ulterior. Am pus această idee în practică prin *instantanee* (snapshots) și chei externe care pot fi anulate, detaliate în Secțiunea 3.3.

2.5 De unde vin regulile analizorului

Analiza structurii unui program pornește de la rezultate din literatura antrenamentului de forță, în special de la relația dintre volumul săptămânal (numărul de serii pe grupă musculară) și creșterea musculară [10]. În aplicație folosesc aceste rezultate ca *reguli* pentru a semnaliza posibile probleme de structură, cum ar fi un volum prea mic sau prea mare ori lipsa unui tipar de mișcare, nu ca rețete individuale. Din acest motiv, analizorul este determinist, explicabil și prudent, iar fiecare rezultat este însoțit de un avertisment despre limitele lui.

2.6 Obiectivele lucrării

Pornind de la acest context, obiectivele lucrării au fost:

1. să construiesc o platformă full-stack funcțională, cu date bine protejate și un model de securitate serios;
2. să realizez un model de citire pentru căutare, separat și eventual consistent, a cărui securitate să nu poată fi ocolită prin parametrii cererii;
3. să verific, pe un set de date în creștere, de la ce dimensiune merită un motor de căutare dedicat față de ce poate face PostgreSQL singur;
4. să verific corectitudinea prin teste de integrare pe infrastructură reală, nu pe substitute.

Capitolul 3

Contribuția proprie

3.1 Cerințe și prezentare generală a sistemului

Pe scurt, un utilizator autentificat își poate gestiona datele de planificare (catalogul comun de exerciții oficiale, exercițiile proprii, rutinele de încălzire/încheiere, sălile și echipamentele), își poate construi *programe* de antrenament pe mai multe zile, poate executa sesiuni live (cu serii, greutate, repetări și RPE), își poate urmări istoricul și progresul, poate publica și copia programe într-un *marketplace* și poate primi o analiză a structurii unui program. În plus, un utilizator cu rol de antrenor poate vedea, doar în citire, datele unui client cu care are o relație activă. Peste programe și peste istoric există și o *căutare* full-text.

Cerințele nefuncționale au influențat toate deciziile importante: PostgreSQL rămâne sursa unică de adevăr; schema este stabilită prin migrări Flyway; fiecare resursă verifică proprietarul, pentru a evita problemele de tip IDOR; iar codul este construit fără substitute în memorie, fără H2 și cu teste de integrare reale. Locurile pregătite pentru capturile din interfață sunt în **Anexa 1**.

3.2 Arhitectura sistemului

Sistemul este o aplicație web pe trei niveluri: o aplicație React single-page comunică prin JSON cu un API REST scris în Spring Boot, iar API-ul persistă datele în PostgreSQL. În producție, frontend-ul compilat este inclus în backend și servit de la aceeași adresă (<http://localhost:8080/>), astfel încât cookie-ul de autentificare rămâne same-origin. Întregul stack poate fi pornit prin Docker Compose (**Anexa 6**).

Backend-ul este organizat *pe funcționalități*, nu după niveluri tehnice globale. Fiecare pachet (`auth`, `exercise`, `routine`, `gym`, `template`, `session`, `history`, `analytics`, `marketplace`, `analyzer`, `search`, `coaching`, `shared`) își are propriile patru niveluri: `domain` (entitățile JPA), `application` (servicii, comenzi, excepții), `infrastructure`

(repository-uri Spring Data și SQL nativ) și **web** (controllere și DTO-uri). Am păstrat strict regula ca nivelul web să nu expună entități JPA, ci doar DTO-uri. Când o funcționalitate are nevoie de datele alteia (de exemplu analizorul citește structura unui program), funcționalitatea care deține datele expune un *model de citire la nivel de aplicație*, nu un DTO web și nici acces direct la repository; în felul acesta dependențele rămân clare.

Autentificarea unei cereri este făcută de un filtru care verifică JWT-ul și pune în contextul de securitate un `UserPrincipal`. Controllerele citesc acest principal și nu se bazează pe un identificador de utilizator primit din corpul cererii. Erorile de domeniu extind o excepție comună, care păstrează atât codul HTTP, cât și un cod stabil de eroare, ușor de tratat de frontend. Un singur handler transformă aceste excepții într-un format uniform (**Anexa 5**). Diagrama de arhitectură este în Figura 3.1.

*[Substituent imagine] Diagrama de arhitectură: SPA React → API
REST Spring Boot → PostgreSQL (sursa de adevăr) și OpenSearch
(model de citire separat).
(se va înlocui cu `\includegraphics` după adăugarea capturii în `images/`)*

Figura 3.1: Arhitectura de ansamblu a sistemului.

3.3 Modelul de date și principiul instantaneelor

Schema este creată prin migrarea inițială **V1**, care acoperă utilizatorii, catalogul de exerciții și grupele musculare, rutinele, sălile și echipamentele, programele cu structura lor (zile / exerciții / rutine), sesiunile live cu instantaneele lor, tabelele de marketplace (voturi, salvări, evenimente de utilizare, statistici per program) și câteva tabele pregătite pentru extinderi ulterioare (coaching, rezultate de analiză, un *outbox*). Migrarea **V2** adaugă exercițiile oficiale, iar **V3** introduce un index pentru concurență. Tabelele și constrângerile sunt sintetizate în **Anexa 2**.

Principiul instantaneelor. Cerința principală este ca istoricul să rămână corect, iar în modelul de date acest lucru este asigurat în două etape. Mai întâi, exercițiile dintr-o zi de program păstrează `exercise_name_snapshot`, `exercise_type_snapshot` și o copie a grupelor musculare, având doar o legătură *care poate fi anulată* către exercițiul viu (`ON DELETE SET NULL`). Apoi, la pornirea unui antrenament, se creează al doilea instantaneu: numele programului, al zilei și al sălii, împreună cu numele/tipul/valorile planificate ale

fiecărui exercițiu și grupele musculare, sunt copiate în `session_exercises`; seriile păstrează și numele echipamentului. Pentru că rândurile sesiunii conțin propriile valori și au doar chei externe anulabile, modificarea sau ștergerea programului, exercițiului, rutinei, sălii ori echipamentului nu schimbă ce s-a notat deja într-o sesiune veche. Acest comportament este verificat direct prin teste.

Ștergere logică, unicitate și constrângeri. Resursele de planificare folosesc o coloană `deleted_at`, astfel încât istoricul care le referă să rămână valid. Unicitatea este impusă prin *indecși unici parțiali* care ignoră rândurile șterse (de exemplu, numele unic al unui exercițiu propriu per utilizator `WHERE deleted_at IS NULL`), ceea ce permite reutilizarea unui nume după ștergere. O parte importantă din reguli este pusă chiar în baza de date prin constrângeri `CHECK`: valorile permise pentru enumerări, `days_per_week BETWEEN 1 AND 7`, `rpe BETWEEN 1.0 AND 10.0`, `finished_at >= started_at`, contoare care nu pot deveni negative, interdicția de a fi propriul antrenor (`coach_user_id <> client_user_id`) și regula că un program public trebuie să aibă o dată de publicare.

Concurență. Regula „un singur antrenament activ per utilizator” este garantată printr-un *index unic parțial* `ux_workout_sessions_one_active_per_user ... WHERE status = 'IN_PROGRESS'` (migrarea V3). Verificarea din serviciu oferă un răspuns mai rapid și mai prietenos, dar garanția finală rămâne în baza de date; dacă două cereri pornesc în același timp, una dintre ele primește un `409 ACTIVE_WORKOUT_EXISTS` controlat, nu o eroare 500. Contoarele de marketplace (voturi, salvări) sunt actualizate prin SQL atomic și mărginit (`GREATEST(0, contor +/- delta)`), evitând cursa citire-modificare-scriere și valorile negative.

3.4 Modelul de securitate

Parolele sunt stocate ca amprente BCrypt. La autentificare, API-ul emite un token de acces JWT de scurtă durată (HS256), pe care aplicația frontend îl trimite în antetul `Authorization` și îl păstrează doar în memorie, precum și un token de reîmprospătare opac, trimis ca un cookie `HttpOnly, SameSite=Strict`, limitat la calea `/api/auth`. Pentru token-ul de reîmprospătare se păstrează în baza de date doar o amprentă SHA-256. La reîmprospătare, token-ul este *rotit*: cel prezentat este verificat și revocat, iar o pereche nouă este emisă în aceeași tranzacție. Dacă un token furat este refolosit, situația poate fi detectată, deoarece amprenta există, dar are `revoked_at` setat. Protecția CSRF este dezactivată intenționat, deoarece token-ul de acces vine din antet, nu din cookie; singurul cookie cu credențiale este cel de reîmprospătare, protejat prin atributele menționate.

Autorizare și siguranță IDOR. Fiecare resursă a unui utilizator este încărcată printr-o interogare care include și identificatorul proprietarului. Dacă un utilizator cere o resursă care aparține altcuiva, răspunsul este **404**, nu **403**, pentru a nu confirma existența resursei. Resursele imbricate verifică proprietarul în lanț; de exemplu, un exercițiu dintr-o zi de program este accesibil doar dacă programul acelei zile aparține utilizatorului. În configurația de securitate, ordinea regulilor este: rute publice de autentificare; `/api/admin/**` doar pentru rolul ADMIN; `/api/coach/**` doar pentru rolul COACH; restul rutelor `/api/**` doar pentru utilizatori autentificați.

3.5 Funcționalitățile aplicației

Date de planificare și programe. Utilizatorii își construiesc o bibliotecă de exerciții (catalogul oficial plus exerciții proprii, fiecare cu grupe musculare primare/secundare), rutine, săli și echipamente. Un program este privat și structurat pe mai multe zile; fiecare zi conține exerciții ordonate (cu serii/repetări/greutate/pauză planificate) și rutine atașate. După modificarea exercițiilor, în PostgreSQL recalculez vectorii agregați ai programului (grupe musculare, identificatori de exerciții oficiale, nume de exerciții), printr-o singură instrucțiune nativă cu `array_agg(DISTINCT ... ORDER BY ...)`, pentru a pregăti căutarea.

Execuția live și istoricul. La pornirea unui antrenament se face, într-o singură tranzacție, copierea-instantaneu descrisă în Secțiunea 3.3. În timpul sesiunii, utilizatorul notează seriile, poate adăuga exerciții neplanificate, iar la final finalizează sau anulează sesiunea. Istoricul afișează mai întâi sesiunile recente și include câteva valori de sumar per sesiune, calculate printr-o singură interogare în lot, pentru a evita problema N+1.

Analize de progres. Analizele sunt calculate în PostgreSQL, folosind sesiunile finalizate: volumul total pe zi, numărul de antrenamente pe săptămână (`date_trunc`), distribuția seriilor pe grupa musculară primară și, pentru graficul de progres la forță, cea mai bună estimare a unei repetări maxime (1RM) pe exercițiu pe zi, folosind formula lui Epley [1]:

$$1RM \approx w \cdot \left(1 + \frac{r}{30}\right), \quad (3.1)$$

unde w este greutatea și r numărul de repetări ale seriei.

Marketplace. Proprietarul poate publica un program nevid, transformându-l într-un program public. Utilizatorii autentificați pot parcurge programele publice (sortate după cele mai noi, scor sau tendință), pot vota, salva și *folosi* un program. Operația de „folosire” face o copie în profunzime a programului public într-un program nou, privat, al utilizatorului care îl copiază. Legătura către exercițiu se păstrează doar dacă exercițiul

este *oficial*; în rest, legătura se anulează, dar se păstrează copia numelui, tipului și grupelelor musculare. Astfel, copia rămâne utilizabilă și independentă de original, fără a expune date private.

3.6 Analizorul structural informat de dovezi

Analizorul este determinist și bazat pe reguli; nu folosește învățare automată. Rezultatul principal este un *Scor de structură a programului*, nu o judecată de „calitate științifică” sau medicală. Fiecare răspuns include un avertisment că analiza nu este sfat medical și că nu ia în calcul factori precum recuperarea, tehnica, nivelul real de efort, somnul, alimentația sau răspunsul individual la antrenament. Analiza se calculează la cerere, printr-un GET read-only (`/api/templates/{id}/analysis`), și este folosită și la indexarea pentru căutare.

Volumul săptămânal este calculat ca sumă peste zilele din program, iar seriile sunt ponderate după rolul muscular (primar 1.0, secundar 0.5). Exercițiile fără serii planificate sunt *excluse* din volum (nu primesc o valoare implicită) și marcate ca date incomplete; doar exercițiile de forță sunt folosite pentru volum și echilibru. Scorul de 100 de puncte este împărțit în cinci părți. Fiecare parte pornește de la maximul ei și scade o valoare fixă pentru fiecare problemă găsită, fără să coboare sub zero, ceea ce face rezultatele deterministe și ușor de testat (Tabelul 3.1). Categoriile sunt **WELL_STRUCTURED** (≥ 80), **DECENT_STRUCTURE** (55–79) și **NEEDS_REVIEW** (< 55). Familiile de reguli și un fragment reprezentativ din logica de scor sunt prezentate în **Anexa 3**.

Sub-scor	Maxim	Exemple de penalizări
Volum și acoperire	35	regiune absentă −12; volum excesiv −5; foarte scăzut −3
Frecvență	20	volum concentrat într-o singură zi −4
Echilibru	20	dezechilibru sever −8; moderat −4
Designul sesiunii	15	sesiune excesivă −5; concentrare pe o grupă −3
Specificitate / pauză	10	pauză prea scurtă −2; interval de repetări extrem −2 ... −3

Tabela 3.1: Descompunerea scorului de structură pe cinci sub-scoruri deterministe.

3.7 Modelul de citire pentru căutare (OpenSearch)

Căutarea este implementată ca un *model de citire separat*, construit peste OpenSearch și activat printr-un indicator de configurare (`app.search.enabled`, implicit oprit, pentru ca dezvoltarea locală și testele pe PostgreSQL să nu depindă de OpenSearch). Sistemul folosește două indexuri, **templates** și **workout_sessions**, cu nume versionate ascunse în spatele unor alias-uri stabile. Analizorul de indexare normalizează textul (lowercase + scoaterea diacriticelor), iar analizorul de căutare adaugă și un set de *sinonime* (de exemplu

pecs→*chest*, *ohp*→*overhead press*). Interogarea `multi_match` aplică ponderi câmpurilor — pentru programe `name^4`, `exerciseNames^3`, `muscleGroupsText^2` etc. — și toleranță la greșeli (`fuzziness=AUTO`), cu o regulă care nu aplică „fuzzy” pentru căutări mai scurte de 3 caractere. În marketplace, scorul de relevanță este combinat cu un *function_score* care introduce o influență logaritmică a popularității (salvări, utilizări). Rezultatele includ fațete (agregări `terms` și histogram pe lună) și evidențierea potrivirilor cu `<mark>`.

Cum scriu în index și cum țin totul consistent. Indexarea este declanșată de evenimente emise *după* salvarea tranzacției (`@TransactionalEventListener(AFTER_COMMIT)`), pentru ca datele anulate prin rollback să nu ajungă în OpenSearch. Există două căi de actualizare: reindexarea *structurală*, care reconstruiește documentul și recalculază câmpurile de analiză, și actualizarea *doar a contoarelor* (vot/salvare/utilizare), care nu recalculează analiza. Scrierile sunt *idempotente*, deoarece documentul este rescris după identificator. Dacă o scriere eșuează, eroarea este jurnalizată și aplicația continuă; PostgreSQL rămâne sursa de adevăr, iar indexul poate fi reparat prin reconstrucție. În forma actuală, soluția *nu* este încă un outbox tranzacțional (vezi Secțiunea 3.11).

Securitate pe două straturi. Primul strat este un *filtru obligatoriu*, derivat din JWT, pe care parametrii cererii nu îl pot suprascrie: `marketplace ⇒ visibility=PUBLIC`; `my ⇒ ownerUserId=apelant`; `workouts ⇒ ownerUserId=apelant`. Al doilea strat este o verificare în PostgreSQL: identificatorii întorși de OpenSearch sunt validați din nou în baza de date, cu aceeași regulă de acces. Astfel, un rezultat învechit (de exemplu, un program abia depus sau șters) este eliminat înainte să ajungă la client, iar pentru marketplace se poate atașa și starea curentă de vot/salvare.

Reconstrucție și reziliență. Serviciul de reconstrucție creează un index nou, cu marcaj de timp, îl umple în lot din PostgreSQL și apoi *schimbă atomic alias-ul*, fără oprirea citirilor. După aceea, indexul vechi este șters; dacă apare o eroare, indexul nou este aruncat, iar cel vechi continuă să servească cererile. Reconstrucția manuală (`POST /api/admin/search/reindex`) este permisă doar rolului ADMIN. Dacă modulul de căutare este oprit sau OpenSearch nu răspunde, endpoint-urile întorc 503, iar interfața revine la lista autoritativă din SQL; un index încă neconstruit este tratat ca „fără rezultate”, nu ca eroare.

Căutarea făcută de antrenor. Un antrenor poate căuta full-text în istoricul unui client (`/api/coach/clients/{id}/search/workouts`). Acesta este singurul caz în care căutarea rulează cu un identificator de proprietar diferit de cel al apelantului. Siguranța vine din faptul că cererea trece mai întâi prin poarta de acces a antrenorului (vezi Secțiunea 3.9) și apoi folosește același serviciu de căutare, dar cu identificatorul clientului,

păstrând filtrul obligatoriu și verificarea din PostgreSQL.

3.8 Evaluarea empirică: OpenSearch vs. PostgreSQL

Partea cea mai importantă din punct de vedere experimental este măsurarea unei întrebări concrete: de la ce dimensiune merită introdus un motor de căutare dedicat, în comparație cu ceea ce poate face PostgreSQL singur?

Metodologie. Varianta PostgreSQL a fost construită cât mai corect, nu ca o variantă slabă pusă să piardă: căutare full-text ponderată cu `tsvector` (`setweight A/B/C/D`) și index GIN, ordonare cu `ts_rank_cd`; căutare fuzzy cu `pg_trgm` și index GIN; fațete prin `GROUP BY`. Partea OpenSearch folosește aceleași mapări și interogări ca în aplicație. Setul de date este generat determinist și *în flux* (pe loturi, cu memorie constantă), cu un vocabular realist de exerciții, popularitate înclinată și distribuție 90%/10% public/privat. Am măsurat șapte dimensiuni, de la 1.000 la 2.000.000 de documente, pe cinci scenarii: două căutări full-text, o căutare fuzzy (greșeală de tastare), o căutare filtrată și o agregare pe fațete. Măsurarea este pe un singur fir, cu încălzire și buget de timp, iar rezultatele sunt raportate prin percentilele $p_{50}/p_{95}/p_{99}$.

Rezultate. Tabelul 3.2 arată latențele la cea mai mare dimensiune (2M de documente), iar tabelele complete, pentru toate dimensiunile, se găsesc în **Anexa 4**. Pe scurt, cele două soluții se echivalează în jurul a *10.000 de documente*: sub această valoare PostgreSQL este mai rapid, iar peste ea OpenSearch câștigă clar. Căutarea full-text în PostgreSQL crește aproape liniar (de la $\sim 1,5$ ms la $\sim 1,86$ s între 1k și 2M), în timp ce OpenSearch rămâne aproape constant (~ 3 – 64 ms). *Căutarea fuzzy este diferența cea mai vizibilă*: `pg_trgm` crește de la 14 ms la $\sim 15,8$ s (1k→2M), față de ~ 4 ms aproape constant în OpenSearch, adică un raport de ordinul a $4000\times$ la 2M. În plus, indexul OpenSearch ocupă de $\sim 3,6\times$ mai puțin pe disc și se construiește de $\sim 2\times$ mai repede la 2M.

Scenariu (la 2M documente)	PostgreSQL	OpenSearch
Full-text „bench press” (p_{50})	1864 ms	64 ms
Full-text „squat” (p_{50})	1437 ms	4 ms
Fuzzy „benhc” (greșeală) (p_{50})	15825 ms	4 ms
Filtrat (press + INTERMEDIATE + PPL) (p_{50})	2295 ms	7 ms
Fațetă după dificultate (p_{50})	1193 ms	2 ms
Timp de construcție a indexului	239 s	115 s
Dimensiune pe disc	2632 MB	739 MB

Tabela 3.2: Comparație OpenSearch vs. PostgreSQL la 2.000.000 de documente. Tabelele complete pe toate cele șapte dimensiuni sunt în Anexa 4.

Interpretare. Rezultatul face mai clară o alegere de arhitectură care este uneori făcută „din reflex”. Sub pragul de echivalență, complexitatea adusă de un motor dedicat este greu de justificat; peste acest prag, mai ales când este nevoie de toleranță la greșeli, motorul dedicat devine o alegere justificată.

3.9 Modul de coaching: RBAC plus ReBAC

Modul de coaching adaugă un strat de control peste RBAC. O relație antrenor–client are un ciclu de viață (`PENDING` → `ACTIVE` dacă este acceptată, sau `REJECTED`; o relație `ACTIVE` poate deveni `REVOKED` la inițiativa oricărei părți), cu un index unic parțial care permite o singură relație vie pentru aceeași pereche. RBAC (`/api/coach/**` cere rolul `COACH`) este necesar, dar nu suficient: fiecare citire pentru un client trece printr-o verificare pe bază de relație (ReBAC), care cere o relație *activă* și, în caz contrar, întoarce **404 (nu 403)**, pentru ca antrenorul să nu poată afla ce utilizatori există. Accesul antrenorului este doar de citire (istoric, analize, o sesiune, căutare); nu există endpoint prin care antrenorul să modifice datele clientului, iar clientul poate retrage accesul oricând.

3.10 Strategia de testare

Corectitudinea este verificată prin teste de integrare pe *infrastructură reală*. O clasă de bază pornește un container PostgreSQL (Testcontainers), iar suita de căutare adaugă un container OpenSearch real și activează indicatorul de căutare; nu folosesc mock-uri sau H2 pentru aceste verificări. Astfel, schema gestionată de Flyway și constrângerile din baza de date sunt testate efectiv. Testele verifică atât comportamentul prin nivelul HTTP, cât și, unde este important, garanțiile bazei de date: o a doua inserare brută a unei sesiuni „în desfășurare” este respinsă de indexul unic parțial; vectorii agregați sunt citați înapoi cu `JdbcTemplate`; instantaneele unei sesiuni rămân neschimbate după editarea datelor sursă. La momentul scrierii, suita are 128 de teste automate și cere un mediu Docker funcțional pentru rularea verificărilor bazate pe Testcontainers.

3.11 Direcții de dezvoltare proiectate

Direcțiile de mai jos sunt *gândite și pregătite*, dar nu sunt încă implementate în cod. Le menționez deoarece arhitectura și, în mai multe cazuri, chiar schema bazei de date le anticipează: schema V1 conține deja, nefolosite de cod, tabelele `outbox_events`, `template_analysis_results`, `coach_session_comments` și `coach_template_assignments`.

- **Cache cu Redis pentru marketplace (cache-aside).** Aș memora paginile de listă și detaliile de program, cu invalidare pornită de aceleași evenimente după

salvare deja existente, plus protecție împotriva efectului de „turmă” prin expirare probabilistică timpurie (XFetch [13]).

- **Outbox tranzacțional pentru indexare.** Ar rezolva problema *dual-write*: un rând de eveniment scris în aceeași tranzacție cu modificarea, în tabela `outbox_events` deja existentă, împreună cu un proces care îl trimite mai departe cu livrare cel-puțin-o-dată și un consumator idempotent (indexatorul este deja idempotent). Aș discuta și alternativa CDC/Debezium [8].
- **Evaluarea calității relevanței.** Aș completa evaluarea de viteză cu o evaluare de calitate: un set de judecăți etichetate și indicatori ca $\text{precision}@k$ și nDCG [3], plus un studiu de ablație (sinonime / fuzzy / ponderi pornite sau oprite).
- **Paginare prin chei (keyset).** Aș înlocui paginarea `OFFSET` de acum cu paginare prin cheie de continuare, ducând costul paginilor adânci de la $\mathcal{O}(n)$ la $\mathcal{O}(\log n)$.
- **Limitarea ratei la autentificare** (token-bucket cu Redis) și **trecerea la Argon2id** pentru parole, cu re-hashing transparent la autentificare.
- **Observabilitate** (trasare distribuită OpenTelemetry + metrice Prometheus/Grafana), **partiționarea pe timp** a tabelii `workout_sessions`, o **coadă de lucru** bazată pe `SELECT ... FOR UPDATE SKIP LOCKED` (potrivită pentru trimiterea din outbox) și **vederi materializate** pentru analize.

Aceste direcții sunt reluate, ca proiectare, în Capitolul 4.

Capitolul 4

Concluzii

4.1 Concluzii privind realizarea temei

Am construit o platformă pentru antrenamente de forță în care integritatea datelor este ideea principală: planurile executate devin istoric care nu se mai schimbă, prin instantanee; analizele și feedback-ul pornesc din date reale; iar marketplace-ul permite partajarea și copierea independentă a programelor. M-am concentrat pe corectitudine (teste pe baze de date reale), pe integritatea datelor (instantanee pe două niveluri, ștergere logică, constrângeri în baza de date), pe securitate (autentificare JWT cu rotație, protecție față de IDOR, RBAC plus ReBAC pentru coaching) și pe explicabilitate (un analizor determinist, cu avertismente), nu pe funcționalități doar decorative.

Căutarea a fost tratată ca o problemă de inginerie a sistemelor: un model de citire separat, eventual consistent și reconstruibil, cu reguli de securitate care nu pot fi ocolite prin parametrii cererii. Cea mai importantă contribuție experimentală este *comparația concretă* dintre OpenSearch și PostgreSQL, realizată pe un set de date crescut până la două milioane de documente. Rezultatele arată că cele două soluții se echivalează în jurul a 10.000 de documente și că diferența cea mai mare apare la căutarea cu greșeli de tastare: varianta din baza de date ajunge la mai multe secunde, în timp ce motorul dedicat rămâne aproape constant. Astfel, o decizie de arhitectură care este uneori luată „din reflex” poate fi susținută prin măsurători.

4.2 Aprecieri personale privind relevanța rezultatelor

Consider că valoarea lucrării nu vine doar din numărul de funcționalități, ci mai ales din felul în care au fost implementate: o singură sursă de adevăr verificată strict, teste de integrare pe infrastructură reală și o măsurare clară a unei alegeri de arhitectură. Rezultatele pot fi folosite și în alte contexte: instantaneele pentru păstrarea istoricului, modelul de citire separat și eventual consistent, precum și metoda de comparare a soluțiilor

de căutare se pot aplica în domenii precum comerț electronic, sisteme de documente sau jurnale de activitate.

4.3 Dezvoltări ulterioare

Arhitectura a fost gândită să poată fi extinsă, iar schema bazei de date pregătește deja mai multe direcții (tabelele `outbox_events`, `template_analysis_results`, `coach_session_comments` și `coach_template_assignments` există din prima migrare, dar nu sunt încă folosite de cod). Pașii următori, descriși ca proiectare în Secțiunea 3.11, ar fi: (1) un strat de cache Redis de tip cache-aside pentru marketplace, cu invalidare condusă de evenimente și protecție împotriva efectului de turmă; (2) un *outbox tranzacțional* pentru indexare, care rezolvă problema dual-write folosind tabela și indexatorul idempotent deja existente; (3) o evaluare a calității relevanței (precision@k , nDCG), care completează măsurarea de viteză deja făcută; (4) paginare prin chei pentru paginile adânci; (5) limitarea ratei la autentificare și trecerea la Argon2id; și (6) observabilitate prin trasare și metrice, partiționarea pe timp a istoricului, o coadă de lucru bazată pe `SKIP LOCKED` și vederi materializate pentru analize. Fiecare dintre aceste direcții extinde ceea ce există deja, fără să schimbe fundamentul sistemului.

Bibliografie

- [1] Boyd Epley, *Poundage Chart. Boyd Epley Workout*, Lincoln, NE: Body Enterprises, 1985.
- [2] Pat Helland, „Immutability Changes Everything”, în *Communications of the ACM* 59.1 (2016), pp. 64–70, DOI: [10.1145/2844112](https://doi.org/10.1145/2844112).
- [3] Kalervo Järvelin și Jaana Kekäläinen, „Cumulated Gain-based Evaluation of IR Techniques”, în *ACM Transactions on Information Systems* 20.4 (2002), pp. 422–446, DOI: [10.1145/582415.582418](https://doi.org/10.1145/582415.582418).
- [4] Michael B. Jones, John Bradley și Nat Sakimura, *JSON Web Token (JWT)*, RFC 7519, Internet Engineering Task Force (IETF), 2015, URL: <https://www.rfc-editor.org/rfc/rfc7519> (accesat în 16.06.2026).
- [5] Martin Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017, ISBN: 978-1449373320.
- [6] Meta Open Source, *React Documentation*, 2024, URL: <https://react.dev/> (accesat în 16.06.2026).
- [7] OpenSearch Project, *OpenSearch Documentation*, 2024, URL: <https://opensearch.org/docs/latest/> (accesat în 16.06.2026).
- [8] Chris Richardson, *Microservices Patterns: With Examples in Java*, Manning Publications, 2018, ISBN: 978-1617294549.
- [9] Stephen Robertson și Hugo Zaragoza, „The Probabilistic Relevance Framework: BM25 and Beyond”, în *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389, DOI: [10.1561/15000000019](https://doi.org/10.1561/15000000019).
- [10] Brad J. Schoenfeld, Dan Ogborn și James W. Krieger, „Dose-response Relationship Between Weekly Resistance Training Volume and Increases in Muscle Mass: A Systematic Review and Meta-analysis”, în *Journal of Sports Sciences* 35.11 (2017), pp. 1073–1082, DOI: [10.1080/02640414.2016.1210197](https://doi.org/10.1080/02640414.2016.1210197).
- [11] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, 2024, URL: <https://www.postgresql.org/docs/16/> (accesat în 16.06.2026).

- [12] The PostgreSQL Global Development Group, *PostgreSQL Documentation: Full Text Search*, 2024, URL: <https://www.postgresql.org/docs/16/textsearch.html> (accesat în 16.06.2026).
- [13] Andrea Vattani, Flavio Chierichetti și Keegan Lowenstein, „Optimal Probabilistic Cache Stampede Prevention”, în *Proceedings of the VLDB Endowment* 8.8 (2015), pp. 886–897, DOI: [10.14778/2757807.2757813](https://doi.org/10.14778/2757807.2757813).
- [14] VMware Tanzu, *Spring Boot Reference Documentation*, 2024, URL: <https://docs.spring.io/spring-boot/index.html> (accesat în 16.06.2026).

Anexa 1 — Capturi de ecran ale interfeței

Această anexă rezervă locuri pentru capturi reprezentative din aplicație. Casetele de mai jos sunt momentan substituenți și vor fi înlocuite cu imaginile reale (`\includegraphics`) după adăugarea lor în directorul `images/`.

*[Substituent imagine] Tabloul de bord (dashboard) după autentificare: un rezumat al activității și legăturile către funcționalități.
(se va înlocui cu `\includegraphics` după adăugarea capturii în `images/`)*

Figura 4.1: Tabloul de bord al utilizatorului.

*[Substituent imagine] Constructorul de programe: zile, exerciții ordonate cu serii/repetări/pauză planificate și rutine atașate.
(se va înlocui cu `\includegraphics` după adăugarea capturii în `images/`)*

Figura 4.2: Constructorul de programe de antrenament.

*[Substituent imagine] Sesiunea live: notarea seriilor (greutate, repetări, RPE) și adăugarea de exerciții în plus.
(se va înlocui cu \includegraphics după adăugarea capturii în images/)*

Figura 4.3: Execuția unei sesiuni de antrenament.

*[Substituent imagine] Analize de progres: volum în timp, antrenamente pe săptămână, distribuția pe grupe musculare și progresia 1RM estimat (grafice Recharts).
(se va înlocui cu \includegraphics după adăugarea capturii în images/)*

Figura 4.4: Pagina de analize și progres.

*[Substituent imagine] Marketplace cu căutare: câmp de căutare, fațete (dificultate, split, grupe musculare) și evidențierea potrivirilor (<mark>).
(se va înlocui cu \includegraphics după adăugarea capturii în images/)*

Figura 4.5: Marketplace-ul de programe cu căutare full-text și fațete.

*[Substituent imagine] Modul de coaching: căutarea full-text în istoricul unui client, accesibilă doar dacă există o relație activă.
(se va înlocui cu \includegraphics după adăugarea capturii în images/)*

Figura 4.6: Căutarea în istoricul clientului (mod coaching).

*[Substituent imagine] Panoul analizorului: scorul pe 100 de puncte,
sub-scorurile și avertismentele, fiecare cu o sugestie.
(se va înlocui cu `\includegraphics` după adăugarea capturii în `images/`)*

Figura 4.7: Rezultatul analizorului structural de programe.

Anexa 2 — Schema bazei de date

Schema este creată de migrarea Flyway V1 (28 de tabele), completată cu date oficiale prin V2 și extinsă de V3 cu un index pentru concurență. Tabelele, grupate pe funcționalități, sunt:

- **Autentificare / coaching:** `app_users`, `refresh_tokens`, `coach_profiles`, `coach_client_relationships`.
- **Catalog / planificare:** `muscle_groups`, `exercises`, `exercise_muscle_groups`, `routines`, `gyms`, `equipment`.
- **Programe:** `workout_templates`, `template_stats`, `template_days`, `template_day_exercises`, `template_day_exercise_muscle_groups`, `template_day_routines`.
- **Sesiuni (istoric care nu se modifică):** `workout_sessions`, `session_exercises`, `session_exercise_muscle_groups`, `session_routines`, `workout_sets`.
- **Marketplace:** `template_votes`, `template_saves`, `template_use_events`.
- **Tabele pregătite pentru extinderi (nefolosite încă de cod):** `coach_session_comments`, `coach_template_assignments`, `template_analysis_results`, `outbox_events`.

Cele mai importante constrângeri de integritate sunt: regula că un program public trebuie să aibă o dată de publicare (`CHECK ((visibility='PUBLIC' AND published_at IS NOT NULL) OR visibility='PRIVATE')`); interdicția de a fi propriul antrenor (`CHECK (coach_user_id <> client_user_id)`); indexul unic parțial pentru un singur antrenament activ per utilizator; indecșii unici parțiali cu `WHERE deleted_at IS NULL`; și cheile externe anulabile (`ON DELETE SET NULL`) către datele sursă, folosite împreună cu valorile de instantaneu. Definițiile-cheie sunt în Anexa 3.

Anexa 3 — Listări de cod reprezentative

Listarea 1. Modul determinist în care lucrează analizorul: fiecare sub-scor pornește de la valoarea maximă și scade o valoare fixă pentru fiecare problemă (extras simplificat din `TemplateAnalyzerService`).

```
private int scoreVolumeAndCoverage() {
    int score = 35;
    // O grupa minora nu poate masca o regiune majora absenta:
    if (!lowerBody) { add("NO_LOWER_BODY", HIGH, ...); score -= 12; }
    if (!pulling)    { add("NO_PULL",      HIGH, ...); score -= 12; }
    if (!pushing)    { add("NO_PUSH",      HIGH, ...); score -= 12; }
    for (String bucket : ALL_BUCKETS) {
        // serii saptamanale ponderate: PRIMARY = 1.0, SECONDARY = 0.5
        double volume = weeklyWeightedSets(bucket);
        if      (volume <= 3) { add("MUSCLE_VOLUME_VERY_LOW", ...); score -= 3; }
        else if (volume > 25) { add("MUSCLE_VOLUME_EXCESSIVE", ...); score -= 5; }
        else if (volume > 20) { add("MUSCLE_VOLUME_HIGH", ...); score -= 2; }
        // ... alte benzi de volum ...
    }
    return Math.max(0, score);
}

private static String category(int score) {
    if (score >= 80) return "WELL_STRUCTURED";
    return score >= 55 ? "DECENT_STRUCTURE" : "NEEDS_REVIEW";
}
```

Listarea 2. Ordinea regulilor de autorizare (din `SecurityConfig`): RBAC pe rute, peste care modulul de coaching adaugă verificarea ReBAC la nivel de serviciu.

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers(POST, "/api/auth/register", "/api/auth/login",
                        "/api/auth/refresh", "/api/auth/logout").permitAll()
    .requestMatchers(GET, "/api/health").permitAll()
    .requestMatchers("/api/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/coach/**").hasRole("COACH")
    .requestMatchers("/api/**").authenticated())
```

```
.anyRequest().permitAll())
```

Listarea 3. Două definiții SQL importante: garda de concurență din V3 și tabela de outbox pregătită încă din V1 pentru extinderea cu outbox tranzacțional.

```
-- Un singur antrenament activ per utilizator (migrarea V3):
CREATE UNIQUE INDEX ux_workout_sessions_one_active_per_user
  ON workout_sessions (user_id) WHERE status = 'IN_PROGRESS';

-- Tabela de outbox, definita in V1, neutilizata inca de cod:
CREATE TABLE outbox_events (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  aggregate_type varchar(100) NOT NULL,
  aggregate_id  uuid NOT NULL,
  event_type    varchar(100) NOT NULL,
  payload       jsonb NOT NULL DEFAULT '{} '::jsonb,
  status        varchar(30) NOT NULL DEFAULT 'PENDING',
  attempts      int NOT NULL DEFAULT 0,
  available_at  timestamptz NOT NULL DEFAULT now(),
  processed_at  timestamptz,
  created_at    timestamptz NOT NULL DEFAULT now(),
  last_error    text,
  CONSTRAINT check_outbox_status
    CHECK (status IN ('PENDING', 'PROCESSING', 'PROCESSED', 'FAILED'))
);
CREATE INDEX outbox_events_processing_idx
  ON outbox_events (status, available_at, created_at);
```

Anexa 4 — Rezultatele complete ale evaluării

Măsurătorile au fost făcute la 7 dimensiuni ale setului de date (1k–2M), cu 300 de iterații măsurate (încălzire 30, buget de timp 15s; n arată câte eşantioane s-au strâns când bugetul a limitat o celulă). Pentru PostgreSQL am folosit `tsvector`+GIN ordonat cu `ts_rank_cd` și fuzzy cu `pg_trgm`; pentru OpenSearch am folosit aceleași mapări și interogări ca în aplicație.

Corpus	Construcție PG	Construcție OS	Dimensiune PG	Dimensiune OS
1.000	216 ms	203 ms	1,6 MB	0,3 MB
10.000	1.926 ms	975 ms	14,6 MB	2,9 MB
50.000	7.966 ms	3.481 ms	68,7 MB	14,3 MB
100.000	14.006 ms	6.349 ms	135,5 MB	28,5 MB
500.000	68.752 ms	37.987 ms	664,7 MB	140,8 MB
1.000.000	125.774 ms	57.701 ms	1.313,6 MB	349,1 MB
2.000.000	239.192 ms	114.908 ms	2.632,5 MB	738,7 MB

Tabela 4.1: Timpul de construcție a indexului și dimensiunea pe disc, pe dimensiuni de corpus.

Corpus	PostgreSQL ($p_{50}/p_{95}/p_{99}$)	OpenSearch ($p_{50}/p_{95}/p_{99}$)
1.000	1,46 / 2,68 / 3,12	3,25 / 5,70 / 10,56
10.000	10,64 / 14,32 / 17,63	2,85 / 5,21 / 7,17
50.000	48,70 / 66,59 / 77,97	5,17 / 10,05 / 13,98
100.000	100,21 / 117,87 / 136,46	6,84 / 10,59 / 12,83
500.000	779,47 / 1021,25 / 1229,16	22,75 / 38,09 / 67,26
1.000.000	872,19 / 1199,43 / 1220,09	33,64 / 45,24 / 83,90
2.000.000	1863,97 / 2783,12 / 2783,12	63,65 / 73,05 / 111,67

Tabela 4.2: Latența căutării full-text „bench press” (ms), pe dimensiuni de corpus.

Corpus	PostgreSQL ($p_{50}/p_{95}/p_{99}$)	OpenSearch ($p_{50}/p_{95}/p_{99}$)
1.000	14,45 / 18,91 / 21,69	2,37 / 3,74 / 5,63
10.000	127,93 / 134,06 / 136,22	3,23 / 7,38 / 10,94
50.000	621,74 / 645,63 / 646,08	2,66 / 4,95 / 6,59
100.000	1831,47 / 2169,61 / 2169,61	3,15 / 5,74 / 7,38
500.000	2447,22 / 8354,43 / 8354,43	4,06 / 7,20 / 8,95
1.000.000	7806,25 / 17790,10 / 17790,10	3,94 / 9,18 / 13,35
2.000.000	15825,49 / 16399,71 / 16399,71	3,89 / 9,38 / 14,14

Tabela 4.3: Latența căutării fuzzy „benhc” (greșeală de tastare, ms): cum se degradează `pg_trgm` față de OpenSearch, care rămâne aproape constant.

Anexa 5 — Contractul API (selecție)

Toate endpoint-urile, cu excepția celor de autentificare și a celui pentru starea de sănătate, cer un token JWT valid. Erorile folosesc un format uniform, cu un cod stabil (de exemplu `TEMPLATE_NOT_FOUND`, `ACTIVE_WORKOUT_EXISTS`, `SEARCH_UNAVAILABLE`).

```
Autentificare : POST /api/auth/{register,login,refresh,logout}; GET /api/auth/me
Planificare  : /api/exercises; /api/routines; /api/gyms;
              /api/gyms/{id}/equipment; /api/equipment/{id}
Programe     : /api/templates; /api/templates/{id};
              POST /api/templates/{id}/{publish,unpublish}
Sesiuni      : POST /api/workouts/start; GET /api/workouts/active;
              POST /api/workouts/{id}/{finish,cancel};
              POST /api/session-exercises/{id}/sets; PUT|DELETE /api/sets/{id}
Istoric      : GET /api/history; GET /api/analytics/overview
Marketplace  : GET /api/marketplace/templates;
              POST /api/marketplace/templates/{id}/{vote,save,use}
Analizor     : GET /api/templates/{id}/analysis
Cautare      : GET /api/search/templates; GET /api/search/workouts;
              POST /api/admin/search/reindex (doar ADMIN)
Coaching     : /api/coach/clients/{id}/{history,analytics,sessions,search}
              (rol COACH + relatie activa);
              /api/coach/clients/invite; DELETE /api/coach/clients/{id};
              /api/coaching/{invites,coaches}
```

Anexa 6 — Configurație și rulare

Întregul stack rulează prin Docker Compose (PostgreSQL 16, OpenSearch 2.13 cu volum persistent și aplicația). Indicatorul `app.search.enabled` controlează pornirea modulului de căutare.

```
# application.yml (extras)
spring:
  jpa:
    hibernate:
      ddl-auto: validate      # schema detinuta de Flyway
  app:
    security:
      jwt:
        issuer: workout-thesis
        access-ttl: 15m
        refresh-ttl: 14d
    search:
      enabled: ${APP_SEARCH_ENABLED:false}
      uri: ${APP_SEARCH_URI:http://localhost:9200}
      templates-index: templates_v1
      sessions-index:  workout_sessions_v1
```

Rularea cu date demonstrative (istoric din 2020 până azi) se face cu profilul `demo`; la pornire, acesta reconstruiește indexul de căutare:

```
SPRING_PROFILES_ACTIVE=demo docker compose up --build
```